

LLD

Credit91 - Low-Level Design

[Credit91 -- Loan Origination & Management System](#)

Next.js 16 + TypeScript + Supabase (Postgres / Auth / Storage) + Vercel

1. Data Model

All tables live in the public schema. profiles is 1:1 with Supabase's built-in auth.users.

profiles

Column	Type	Notes
id	uuid, PK	= auth.users.id
role	text	'applicant' 'underwriter'
full_name	text	
created_at	timestampz	default now()

applications

Column	Type	Notes
id	uuid, PK	default gen_random_uuid()
user_id	uuid, FK -> profiles	
status	text	submitted approved rejected accepted disbursed
income	numeric(14,2)	
employment_info	text	
requested_amount	numeric(14,2)	
requested_tenure_months	int	> 0
created_at / updated_at	timestampz	

documents, credit_checks

Table	Key columns
documents	id, application_id (FK), file_url, type, uploaded_at
credit_checks	id, application_id (FK), score (300-900), checked_at

decisions

Column	Type	Notes
application_id	uuid, FK, UNIQUE	one decision per application
decision	text	'approved' 'rejected'
approved_amount / interest_rate / tenure_months	numeric / int	set only when approved
decided_by	uuid, FK -> profiles	underwriter
reason	text	required by UI when rejecting

loans, repayment_schedule, payments

Table	Key columns
loans	id, application_id (FK, UNIQUE), principal, interest_rate, tenure_months, outstanding_balance, status (active delinquent closed), disbursed_at
repayment_schedule	id, loan_id (FK), installment_no, due_date, amount_due, principal_component, interest_component, status (due paid overdue). UNIQUE(loan_id, installment_no)
payments	id, loan_id (FK), schedule_id (FK), amount, paid_at

audit_log

id, entity, entity_id, action, actor_id, details (jsonb), created_at -- one row inserted on every key action (submit, credit check, decision, upload, disbursement, payment, accrual).

2. Core Data Structures

This codebase is functional TypeScript (React Server Components + Route Handlers), not class-based -- the closest equivalent to "class structures" is the shared type contract in `lib/types.ts`, which every API route and page imports so the shape of a row is defined once and checked at compile time.

Domain enums

```
type Role = "applicant" | "underwriter"
type ApplicationStatus = "submitted" | "approved" | "rejected"
                        | "accepted" | "disbursed"
type DecisionType = "approved" | "rejected"
type LoanStatus = "active" | "delinquent" | "closed"
type ScheduleStatus = "due" | "paid" | "overdue"
```

Domain interfaces (1:1 with the tables above)

```
interface Application { id, user_id, status: ApplicationStatus, income,
  employment_info, requested_amount, requested_tenure_months,
  created_at, updated_at }

interface Decision { id, application_id, decision: DecisionType,
  approved_amount, interest_rate, tenure_months, decided_by, reason, decided_at }

interface Loan { id, application_id, principal, interest_rate, tenure_months,
  outstanding_balance, status: LoanStatus, disbursed_at }

interface RepaymentInstallment { id, loan_id, installment_no, due_date,
  amount_due, principal_component, interest_component, status: ScheduleStatus }
```

The one piece of real domain logic that behaves like a class in spirit -- encapsulated state transformation with no side effects -- is `generateAmortizationSchedule(principal, annualRatePercent, tenureMonths, startDate)` in `lib/amortization.ts`: a pure function that takes a loan's terms and returns its full installment schedule, deliberately kept free of I/O so it is trivially unit-testable (see Testing Strategy).

3. State Machines

Application.status

```
submitted -> approved -> accepted -> disbursed  
          -> rejected (terminal)
```

'accepted' and 'disbursed' collapse into a single step today: accepting an offer disburses immediately since there is no real payment rail to wait on. In production these would separate, with 'accepted' waiting on a disbursement-confirmation webhook before moving to 'disbursed'.

Loan.status

```
active -> delinquent (has one or more overdue installments)  
active -> closed      (every installment paid)
```

repayment_schedule.status

```
due -> paid      (a payment is simulated against it)  
due -> overdue   (due_date has passed with no payment, flagged by Run Accrual)
```

4. API Specification

Method	Route	Notes
POST	/api/applications	applicant creates an application
GET	/api/applications	list -- own rows (applicant) or all (underwriter)
GET	/api/applications/[id]	single application with joins
POST	/api/applications/[id]/documents	multipart upload to Supabase Storage
POST	/api/applications/[id]/credit-check	mock score generator
PATCH	/api/applications/[id]/decision	underwriter only: approve/reject
POST	/api/applications/[id]/accept-offer	creates Loan + schedule
GET	/api/loans/[id]/schedule	schedule + payments for a loan
POST	/api/loans/[id]/payments	simulate paying next/specific installment
POST	/api/loans/[id]/run-accrual	underwriter only: flag overdue + apply penalty
GET	/api/admin/collections	underwriter only: loans with a missed due date

Auth itself (signUp, signInWithPassword, signOut) goes directly through the Supabase client SDK from the browser -- there are no custom /api/auth/ routes.*

5. Core Algorithms

Amortization (lib/amortization.ts)

Standard reducing-balance EMI. Given principal P, nominal annual rate annualPct, and tenure n months:

```
monthlyRate r = annualPct / 100 / 12
EMI = P * r * (1+r)^n / ((1+r)^n - 1)          (r = 0 => EMI = P / n)

For each installment i = 1..n:
  interest_i = round(balance * r)
  principal_i = min(EMI - interest_i, balance)    (last installment = remaining
balance)
  amount_due = principal_i + interest_i
  balance -= principal_i
```

All money math is done in integer cents to avoid floating-point drift, and the final installment absorbs any rounding remainder so the balance always reaches exactly zero. Verified example (tested against this build): P=100,000, annual rate 10%, 12 months -> EMI = 8,791.59 for installments 1-11, and 8,791.56 for installment 12 (absorbing the 3-cent rounding remainder). Covered by 7 unit tests in lib/amortization.test.ts, including a hand/calculator-verified EMI check, a 0% interest edge case, and a full-amortization balance check.

Interest accrual (POST /api/loans/[id]/run-accrual)

For the given loan, find every repayment_schedule row with due_date in the past and status in (due, overdue). For each: apply a 2% late-payment penalty on that installment's amount_due, add it to the loan's outstanding_balance, and mark the row 'overdue'. Mark the loan 'delinquent'. Verified example: a loan with 2 overdue installments and an INR 60,000.00 balance moved to INR 60,416.50 after one run (INR 208.25 penalty per installment x 2).

Payment simulation (POST /api/loans/[id]/payments)

Selects the earliest un-paid installment (or a specific scheduleId), inserts a payments row for its full amount_due, marks the installment 'paid', and reduces the loan's outstanding_balance by that installment's principal_component only (the interest portion is not principal, so it does not reduce the balance further). If no installments remain unpaid, the loan is marked 'closed'.

6. Row Level Security Summary

A SECURITY DEFINER helper function `my_role()` reads the caller's role from profiles without re-triggering RLS recursively. Every table's policies follow the same shape:

```
USING ( <row belongs to auth.uid()> OR my_role() = 'underwriter' )
```

- profiles: read own row, or any row if underwriter (needed to show applicant names on the underwriter dashboard).
- applications / documents / credit_checks / decisions / loans / repayment_schedule / payments: visible/writable if the row traces back (directly or via a join to applications) to the caller's own `user_id`, or the caller is an underwriter.
- audit_log: any authenticated user may insert their own action; only underwriters may read it.
- storage.objects (bucket 'documents'): files are stored under ``${application_id}/filename``; the same ownership check is applied by parsing the folder name back to an applications row.

7. Error Handling & Validation

- Every API route returns a consistent JSON error shape (`{ error: string }`) with an appropriate HTTP status via a shared `jsonError()` helper (`lib/http.ts`): 401 unauthenticated, 403 wrong role or not the owner, 400 invalid input or wrong application/loan state, 404 not found.
- Ownership and state checks happen before any mutation -- e.g. `accept-offer` verifies `application.user_id` matches the caller and `application.status` is `'approved'` before a `Loan` row is ever created.
- Client-side forms use HTML5 `required/min/type` constraints as a first pass for a fast UX, but the API route is the source of truth: nothing sent by the client is trusted for authorization or state transitions.
- Postgres `CHECK` constraints are the final backstop independent of application code -- e.g. `requested_tenure_months > 0`, `credit_checks.score` between 300 and 900, `status` columns constrained to their enum values.

8. Testing Strategy

- Unit tests (lib/amortization.test.ts, 7 cases): the single highest-risk piece of business logic -- a hand/calculator-verified EMI check, a 0% interest edge case, a full-amortization balance-reaches-zero check, and input validation.
- Manual end-to-end verification against the live deployment and a real Postgres database (not mocks): the full applicant -> underwriter -> disbursement -> repayment -> accrual -> collections flow, confirming RLS policies, PostgREST relationship embedding, and the amortization/accrual math all behave correctly together, not just in isolation.
- Not yet covered, called out honestly: an automated integration/E2E suite (e.g. Playwright) exercising the API routes and UI. If extending this, the accept-offer -> disbursement path and the accrual calculation are the highest-value targets to automate first, since both are money-critical and hard to eyeball-verify once in production.